# README

## README

## Lab_Thread_Counter — Static vs Non-Static Counter in Multithreading

**Name:** Pritom Kumar Sen Anom | **ID:** IT24061 | **Course:** SEPM-001

### 1. Files

- `Anom_Thread.java` – typed version (for running)
- `handwritten/` – photo/scan of handwritten code
- `outputs/` – screenshots of every run (+ raw `.txt`, `results.csv`)
- `run_all.sh` – script that runs all test cases

### 2. How to run

```
javac Anom_Thread.java
java Anom_Thread 10 100000 true    # thread-safe (AtomicLong)
java Anom_Thread 10 100000 false   # unsynchronized
```

### 3. Test inputs

| Test | Threads | Increments/thread | Expected |
|---|---|---|---|
| TC1 | 1 | 1,000 | 1,000 |
| TC2 | 2 | 10,000 | 20,000 |
| TC3 | 5 | 10,000 | 50,000 |
| TC4 | 10 | 50,000 | 500,000 |
| TC5 | 20 | 50,000 | 1,000,000 |
| TC6 | 50 | 50,000 | 2,500,000 |
| TC7 | 100 | 50,000 | 5,000,000 |

### 4. Output screenshots

Screenshots are from separate runs of the program. The unsynchronized values change between runs, so they differ slightly from the tables in Section 5 (see Section 6).

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter> powershell -Execution
Policy Bypass -File .\show_cases.ps1 1

===== TC1: java Anom_Thread 1 1000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 1
Increments/thr   : 1000
Expected count   : 1000
Static count     : 1000
Non-static total: 1000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC1: java Anom_Thread 1 1000 false =====
Mode             : Unsynchronized (long)
Threads          : 1
Increments/thr   : 1000
Expected count   : 1000
Static count     : 1000
Non-static total: 1000
Absolute diff    : 0
Difference (%)   : 0.0000
```

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter>  powershell -Executio
nPolicy Bypass -File .\show_cases.ps1 2

===== TC2: java Anom_Thread 2 10000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 2
Increments/thr   : 10000
Expected count   : 20000
Static count     : 20000
Non-static total: 20000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC2: java Anom_Thread 2 10000 false =====
Mode             : Unsynchronized (long)
Threads          : 2
Increments/thr   : 10000
Expected count   : 20000
Static count     : 11420
Non-static total: 20000
Absolute diff    : 8580
Difference (%)   : 42.9000
```

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter> powershell -Execution
Policy Bypass -File .\show_cases.ps1 3

===== TC3: java Anom_Thread 5 10000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 5
Increments/thr   : 10000
Expected count   : 50000
Static count     : 50000
Non-static total: 50000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC3: java Anom_Thread 5 10000 false =====
Mode             : Unsynchronized (long)
Threads          : 5
Increments/thr   : 10000
Expected count   : 50000
Static count     : 13428
Non-static total: 50000
Absolute diff    : 36572
Difference (%)   : 73.1440
```

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter> powershell -Execution
Policy Bypass -File .\show_cases.ps1 4

===== TC4: java Anom_Thread 10 50000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 10
Increments/thr   : 50000
Expected count   : 500000
Static count     : 500000
Non-static total: 500000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC4: java Anom_Thread 10 50000 false =====
Mode             : Unsynchronized (long)
Threads          : 10
Increments/thr   : 50000
Expected count   : 500000
Static count     : 121260
Non-static total: 500000
Absolute diff    : 378740
Difference (%)   : 75.7480
```

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter> powershell -Execution
Policy Bypass -File .\show_cases.ps1 5

===== TC5: java Anom_Thread 20 50000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 20
Increments/thr   : 50000
Expected count   : 1000000
Static count     : 1000000
Non-static total: 1000000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC5: java Anom_Thread 20 50000 false =====
Mode             : Unsynchronized (long)
Threads          : 20
Increments/thr   : 50000
Expected count   : 1000000
Static count     : 370014
Non-static total: 1000000
Absolute diff    : 629986
Difference (%)   : 62.9986
```

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter> powershell -Execution
Policy Bypass -File .\show_cases.ps1 6

===== TC6: java Anom_Thread 50 50000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 50
Increments/thr   : 50000
Expected count   : 2500000
Static count     : 2500000
Non-static total: 2500000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC6: java Anom_Thread 50 50000 false =====
Mode             : Unsynchronized (long)
Threads          : 50
Increments/thr   : 50000
Expected count   : 2500000
Static count     : 1195648
Non-static total: 2500000
Absolute diff    : 1304352
Difference (%)   : 52.1741
```

```
PS C:\Users\AC\Desktop\Lab_Thread_Counter> powershell -Execution
Policy Bypass -File .\show_cases.ps1 7

===== TC7: java Anom_Thread 100 50000 true =====
Mode             : Thread-safe (AtomicLong)
Threads          : 100
Increments/thr   : 50000
Expected count   : 5000000
Static count     : 5000000
Non-static total: 5000000
Absolute diff    : 0
Difference (%)   : 0.0000

===== TC7: java Anom_Thread 100 50000 false =====
Mode             : Unsynchronized (long)
Threads          : 100
Increments/thr   : 50000
Expected count   : 5000000
Static count     : 3133108
Non-static total: 5000000
Absolute diff    : 1866892
Difference (%)   : 37.3378
```

# 5. Result analysis

## Thread-safe (AtomicLong)

| Threads | Increments/thread | Expected | Static | Non-static | Abs diff | Diff % |
|---|---|---|---|---|---|---|
| 1 | 1,000 | 1,000 | 1,000 | 1,000 | 0 | 0.0000 |
| 2 | 10,000 | 20,000 | 20,000 | 20,000 | 0 | 0.0000 |
| 5 | 10,000 | 50,000 | 50,000 | 50,000 | 0 | 0.0000 |
| 10 | 50,000 | 500,000 | 500,000 | 500,000 | 0 | 0.0000 |
| 20 | 50,000 | 1,000,000 | 1,000,000 | 1,000,000 | 0 | 0.0000 |
| 50 | 50,000 | 2,500,000 | 2,500,000 | 2,500,000 | 0 | 0.0000 |
| 100 | 50,000 | 5,000,000 | 5,000,000 | 5,000,000 | 0 | 0.0000 |

## Unsynchronized (5 runs each)

| Threads | Run | Expected | Static (unsafe) | Non-static | Abs diff | Diff % |
|---|---|---|---|---|---|---|
| 1 | 1 | 1,000 | 1,000 | 1,000 | 0 | 0.00 |
| 1 | 2 | 1,000 | 1,000 | 1,000 | 0 | 0.00 |
| 1 | 3 | 1,000 | 1,000 | 1,000 | 0 | 0.00 |
| 1 | 4 | 1,000 | 1,000 | 1,000 | 0 | 0.00 |
| 1 | 5 | 1,000 | 1,000 | 1,000 | 0 | 0.00 |
| 2 | 1 | 20,000 | 12,454 | 20,000 | 7,546 | 37.73 |
| 2 | 2 | 20,000 | 10,110 | 20,000 | 9,890 | 49.45 |
| 2 | 3 | 20,000 | 11,547 | 20,000 | 8,453 | 42.27 |
| 2 | 4 | 20,000 | 12,387 | 20,000 | 7,613 | 38.06 |
| 2 | 5 | 20,000 | 13,350 | 20,000 | 6,650 | 33.25 |
| 5 | 1 | 50,000 | 14,073 | 50,000 | 35,927 | 71.85 |
| 5 | 2 | 50,000 | 15,121 | 50,000 | 34,879 | 69.76 |
| 5 | 3 | 50,000 | 13,732 | 50,000 | 36,268 | 72.54 |
| 5 | 4 | 50,000 | 14,640 | 50,000 | 35,360 | 70.72 |
| 5 | 5 | 50,000 | 14,923 | 50,000 | 35,077 | 70.15 |
| 10 | 1 | 500,000 | 106,521 | 500,000 | 393,479 | 78.70 |
| 10 | 2 | 500,000 | 104,498 | 500,000 | 395,502 | 79.10 |
| 10 | 3 | 500,000 | 124,176 | 500,000 | 375,824 | 75.16 |
| 10 | 4 | 500,000 | 144,325 | 500,000 | 355,675 | 71.14 |
| 10 | 5 | 500,000 | 124,629 | 500,000 | 375,371 | 75.07 |
| 20 | 1 | 1,000,000 | 263,966 | 1,000,000 | 736,034 | 73.60 |
| 20 | 2 | 1,000,000 | 360,635 | 1,000,000 | 639,365 | 63.94 |
| 20 | 3 | 1,000,000 | 243,490 | 1,000,000 | 756,510 | 75.65 |
| 20 | 4 | 1,000,000 | 289,439 | 1,000,000 | 710,561 | 71.06 |
| 20 | 5 | 1,000,000 | 224,590 | 1,000,000 | 775,410 | 77.54 |
| 50 | 1 | 2,500,000 | 820,104 | 2,500,000 | 1,679,896 | 67.20 |
| 50 | 2 | 2,500,000 | 545,572 | 2,500,000 | 1,954,428 | 78.18 |
| 50 | 3 | 2,500,000 | 1,368,824 | 2,500,000 | 1,131,176 | 45.25 |
| 50 | 4 | 2,500,000 | 1,291,877 | 2,500,000 | 1,208,123 | 48.32 |
| 50 | 5 | 2,500,000 | 1,181,041 | 2,500,000 | 1,318,959 | 52.76 |
| 100 | 1 | 5,000,000 | 3,354,875 | 5,000,000 | 1,645,125 | 32.90 |

| Threads | Run | Expected | Static (unsafe) | Non-static | Abs diff | Diff % |
|---|---|---|---|---|---|---|
| 100 | 2 | 5,000,000 | 3,310,492 | 5,000,000 | 1,689,508 | 33.79 |
| 100 | 3 | 5,000,000 | 3,316,579 | 5,000,000 | 1,683,421 | 33.67 |
| 100 | 4 | 5,000,000 | 2,992,003 | 5,000,000 | 2,007,997 | 40.16 |
| 100 | 5 | 5,000,000 | 3,471,530 | 5,000,000 | 1,528,470 | 30.57 |

### Average of 5 unsynchronized runs

| Threads | Avg static | Avg abs diff | Avg Diff % | Min % | Max % |
|---|---|---|---|---|---|
| 1 | 1,000 | 0 | 0.00 | 0.00 | 0.00 |
| 2 | 11,970 | 8,030 | 40.15 | 33.25 | 49.45 |
| 5 | 14,498 | 35,502 | 71.00 | 69.76 | 72.54 |
| 10 | 120,830 | 379,170 | 75.83 | 71.14 | 79.10 |
| 20 | 276,424 | 723,576 | 72.36 | 63.94 | 77.54 |
| 50 | 1,041,484 | 1,458,516 | 58.34 | 45.25 | 78.18 |
| 100 | 3,289,096 | 1,710,904 | 34.22 | 30.57 | 40.16 |

## 6. Observation & Conclusion

### Observations

**Thread-safe experiment (AtomicLong).** In all seven test cases (1 to 100 threads) the static counter matched the non-static total exactly. The absolute difference was 0 and the percentage difference was 0%. This is the control case and confirms that the program counts correctly when the shared counter is updated atomically.

**Unsynchronized experiment (regular `long`).** The non-static total was always exactly correct, because each thread increments its own private counter. The static counter, however, was lower than expected in every run with 2 or more threads, so increments were lost.

| Threads | Average difference (%) |
|---|---|
| 1 | 0.00 |
| 2 | 40.15 |
| 5 | 71.00 |
| 10 | 75.83 |
| 20 | 72.36 |
| 50 | 58.34 |
| 100 | 34.22 |

- **1 thread:** no loss (0%), because only one thread touches the counter, so no race is possible.
- **2 to 10 threads:** the average difference rose from about 40% to about 76%. With more threads, it is more likely that several threads read the same old value at the same time, so more updates overwrite each other.
- **20 to 100 threads:** the percentage difference fell (72% at 20, 58% at 50, 34% at 100), even though the absolute number of lost increments kept growing (about 724,000 at 20 threads, 1,459,000 at 50 and 1,711,000 at 100). The expected total grows faster than the number of lost updates, so the percentage drops. A likely additional reason is that when there are far more threads than CPU cores, the OS runs them in turns, so fewer threads truly execute at the same instant. The exact cause on this machine was not measured, so this is a probable explanation, not a proven one.
- **Variation between runs:** for the same thread count the results differed noticeably. At 50 threads the difference ranged from 45.25% to 78.18%. At 20 threads it ranged from 63.94% to 77.54%. Thread scheduling is decided by the OS and JVM and changes from run to run, so the lost updates are not repeatable.

### Why this happens

`count++` on a normal `long` is not one operation. It is three steps: read the value, add 1, write it back. If two threads read the same value before either writes, both write the same result and one increment is lost. This is a race condition. `AtomicLong.incrementAndGet()` performs the whole update as one indivisible operation (using compare-and-set), so no update is lost. The non-static counter is never shared, so it needs no protection.

### Conclusion

- `static` versus non-static describes **ownership** (one copy for the class versus one copy per object). It does not make a variable thread-safe.
- A shared static counter needs synchronization or atomic operations; otherwise concurrent updates are lost.
- More threads can cause lost updates, but the percentage difference does **not** increase steadily with thread count. In these experiments it rose up to 10 threads and then fell, depending on scheduling, hardware and runtime conditions.
- The thread-safe experiment gave 0% difference in every case, as expected.

## 7. Analysis questions (answers)

1. Static belongs to the class (one copy); non-static belongs to each object.
2. Because there is only one copy per class, every thread accesses the same memory.
3. Each thread gets its own object, so its own instanceCount copy.
4. join() makes main wait; otherwise totals are read before threads finish.
5. Lost updates from the read-modify-write race on `count++`.
6. No - depends on scheduling/hardware; trend is not guaranteed monotonic.
7. Different interleavings each run.
8. AtomicLong uses atomic CAS, so each increment is indivisible → no lost updates.
9. Make the instance counter `static` like shared: create one object and pass the same object to all `Thread`s (and make its counter AtomicLong/synchronized).